

15/2/24

#

Date:

Page No.

Cache Mapping -Associative Mapping -

CPU - address (15 bits)

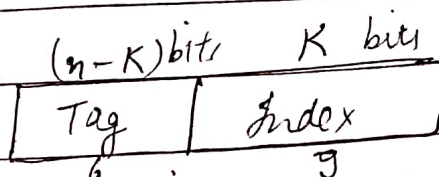
↓

Argument Reg.

address (15)	Data (n)
01000	3450
02777	6710
22345	0173

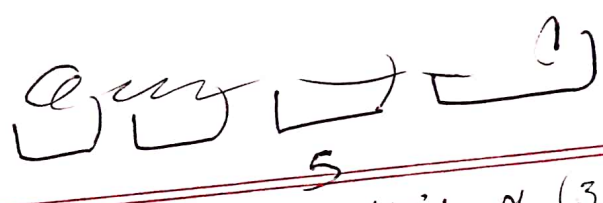
In this mapping, any location in cache stores any word from main Memory.

Cache replacement → when cache is full, is done through FIFO or LRU etc.

~ Direct Mapping -Direct Mapping

CPU address = n bits (15)

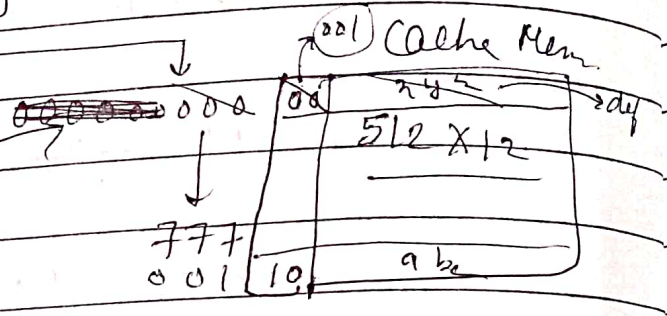
~~(n-K) bits~~



since 15 bits $\equiv$ multiple of (3)

Tag	Index
00	1000

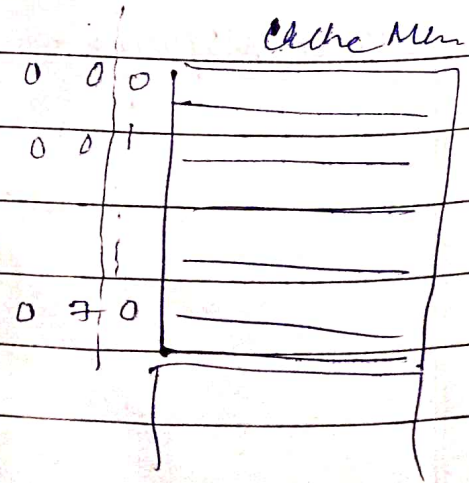
00	000	Main 32Kx12 Memory	
00	777		
01	000		def
10	001		abc
77	777		



Cache with 8 word block size -

512x12
Number of 8 word block = 64 (64x8 = 512)

6	6	3
Tag	block	word



Index { Main Memory }

Date:

Page No.

Block	word	Tag	word
00	0	010	3450
00	7	01	6571
01	0		
01	7		
77	0	02	5210
77	7	02	6710

~ Cache write Policies -

write through Policy → slower
write back Policy → faster

→ update the memory with every Memory-write operation.

- Main Mem always contains the same data as the cache.

- Important for DMA, so that IO device receives most recent data from main Memory.

(DMA)

PIO
process

CPU

Main
Mem.

IO devices

direct copy of data to Mem.

but

advantage

as in Main

Mem. More

recent data is in
Main Mem.

↓

more helpful to
give correct value
in process switching.

(Context)
- Multiprogramming - process switching

- ② - Only cache location is updated & flagged {dirty bits}
 - Mem. is updated when word is removed from cache.
 - bcz cache's word may be updated many times & as long as data is in cache, CPU request for that data are satisfied from the cache.

- This is faster than ①

Set associative Mapping -

It's an improvement

on direct Mapping.

Tag cache data Tag Data

000 00 3450 01 5326

←6→ ←12→ ←6→ ←12→

777

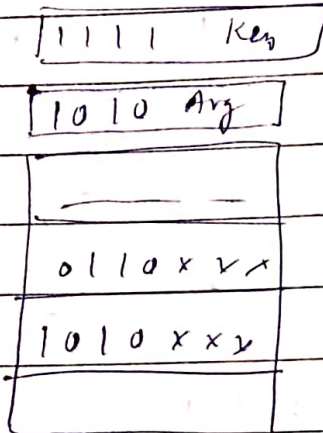
←36→
word size

* Cache Initialization -

valid bit is set to 1. when the word is entered for the first time in cache. At Initialization, all valid bits are set to 0, so when a word is accessed whose valid bit is found to be 0, the new word automatically replaces the invalid data.

- * A valid word will be replaced only when a mismatch of tags occurs in direct mapping.
{ word is valid when valid bit is 1 }

27/02/24

Associative Memory & Content addressable Memory

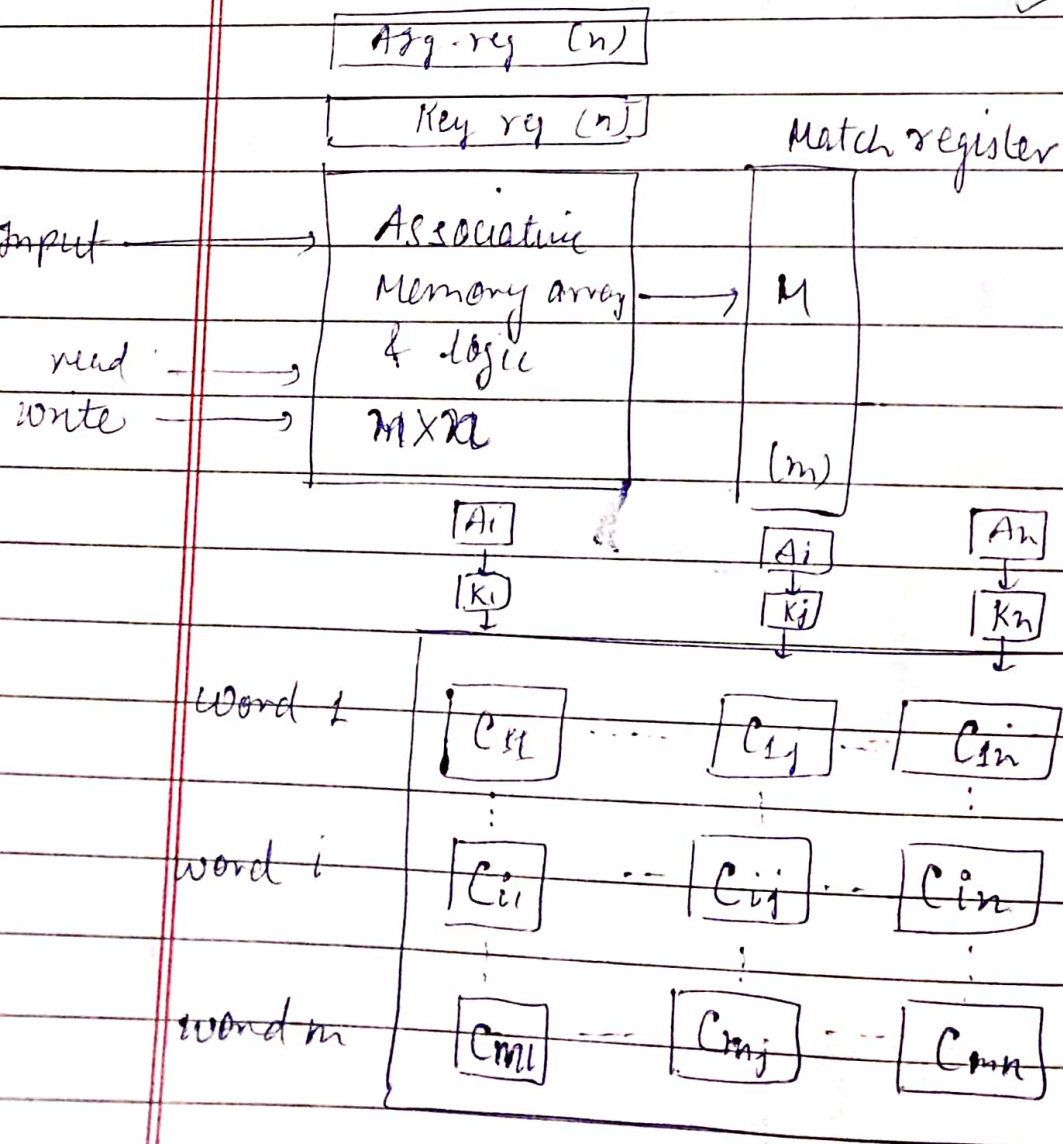
To be matched

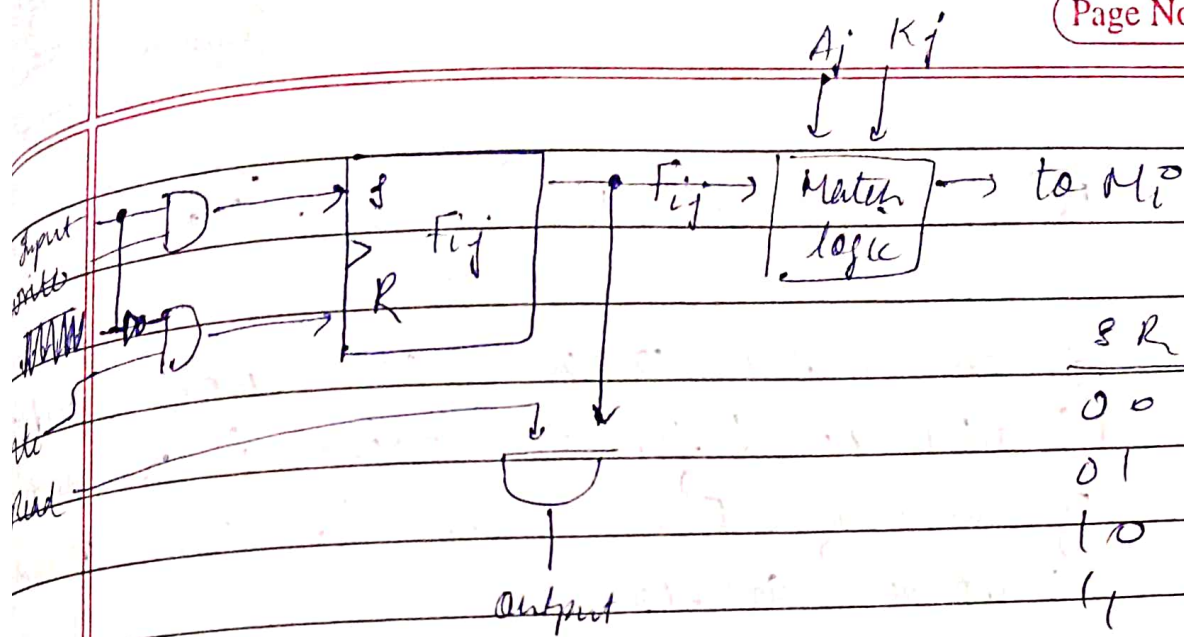
$$A = 10111101$$

$$K = 11100000$$

$$\text{word 1} = \frac{100}{x} 111101$$

$$\text{word 2} = \frac{10}{\checkmark} 000110$$





S	R	Output
0	0	0
0	1	0
1	0	1
1	1	N.D

~ Match logic -

a word is matched to arg. A if
(we neglect K here).

$$A_j = F_{ij} \quad \text{for all } j = 1 \text{ to } n$$

or $x_j = A_j \odot F_{ij} = 1$
for all $j = 1 \text{ to } n$

Thus for bits of A & word i

$$M_i = x_1 \cdot x_2 \cdot \dots \cdot x_n = 1$$

~ let's include K in matching function.

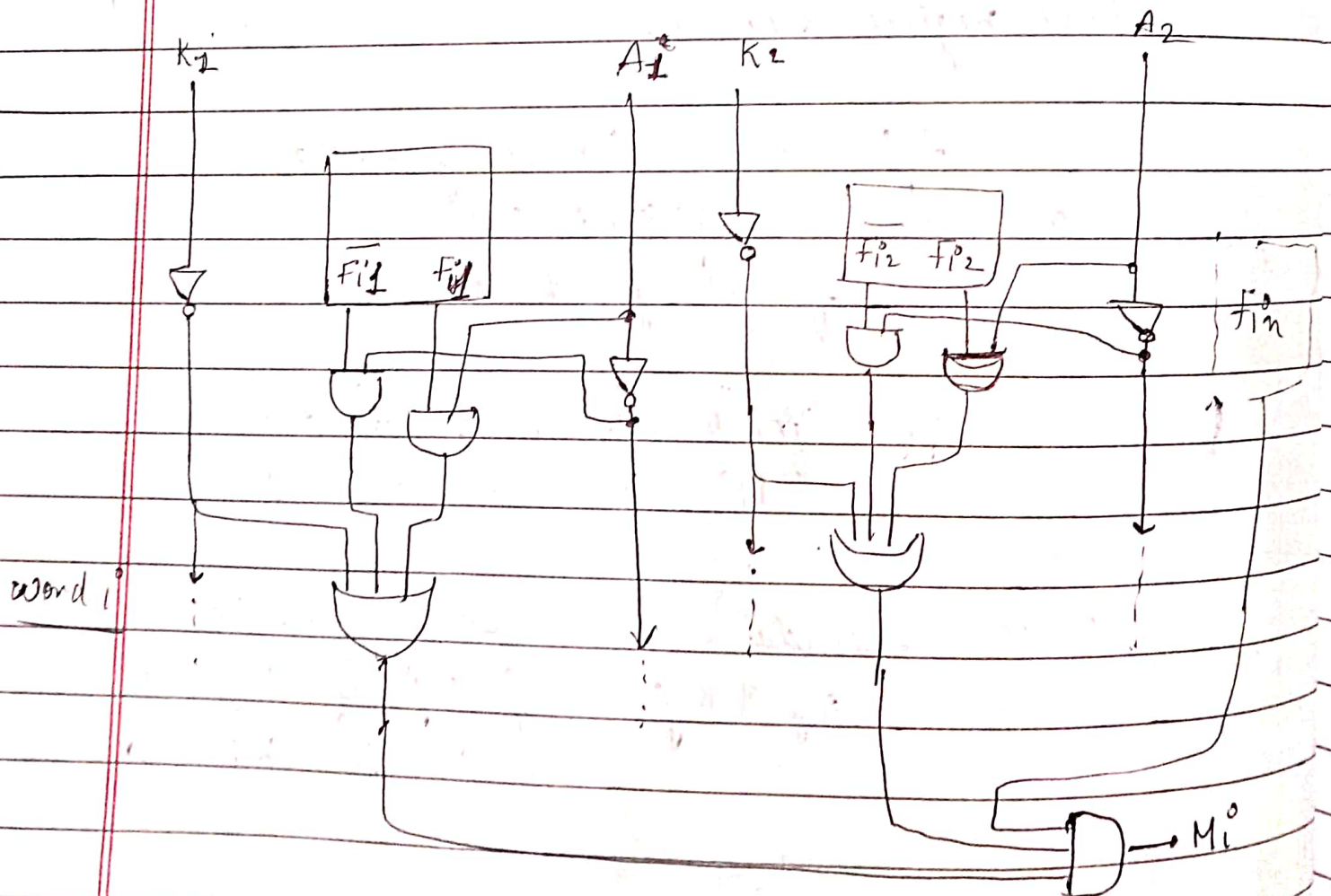
$$x_j \cdot K_j = \begin{cases} x_j & \text{if } K_j = 1 \\ 1 & \text{if } K_j = 0 \end{cases}$$

$$M_i^0 = (x_1 + K_1')(x_2 + K_2') \dots (x_n + K_n')$$

If Key reg. K contains all zeros, output M_i^0 will be 1 for each word i irrespective of value of $E^0 A$ or word i . This occurrence must be avoided

$$M_i^0 = \bigwedge_{j=1}^n (A_j^0 \cdot F_{ij}^0 + \overline{A_j^0} \cdot \overline{F_{ij}^0} + K_j^0)$$

$\forall i^0 = 1, \dots, m$



word $(i+1)$

The read operation -

① If more than 1 word in memory matches the arguments, all the matched words will have 1's in corresponding M_i .

② The matched reg. is scanned 1 bit at a time, and matched words are read in sequence, by applying read signal to those words where corresponding M_i is 1.

③ Generally associative memory stores no two identical items under a given key. Thus only 1 word may match ~~unmasked~~ unmasked argument field. Here M_i is not sent to reg. M rather it is directly connected to the read signal of the word i . Thus, the content will be presented automatically at the output lines and no special read signal is needed.

~ write operation - If all the information has to be entered at once prior to a search operation, each location can be addressed sequentially like a random access memory.
i.e. m words of memory can be accessed by

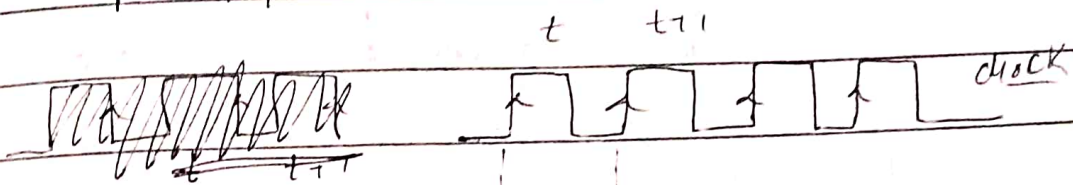
d lines and a decoder such that
 $m = 2^d$.

No. of address lines are reduced from
 $m \rightarrow d$.

~ If unwanted words have to be deleted and new words inserted one at a time, a new register called a tag register can be used. Tag reg. has one bit for each word of memory. Tag bit is 1 if word is active. A word is deleted from memory by clearing its tag bit to zero. Words are stored in memory by scanning the tag register and inserting at the first word with tag bit = 0. After a new word is stored in memory, the tag bit is set to 1.

	unmasked	masked
mask	1111	0000
	1010	1111
	1010	0000

1/3/24

microoperation (uops) : $P: R_1 \leftarrow R_2$ 

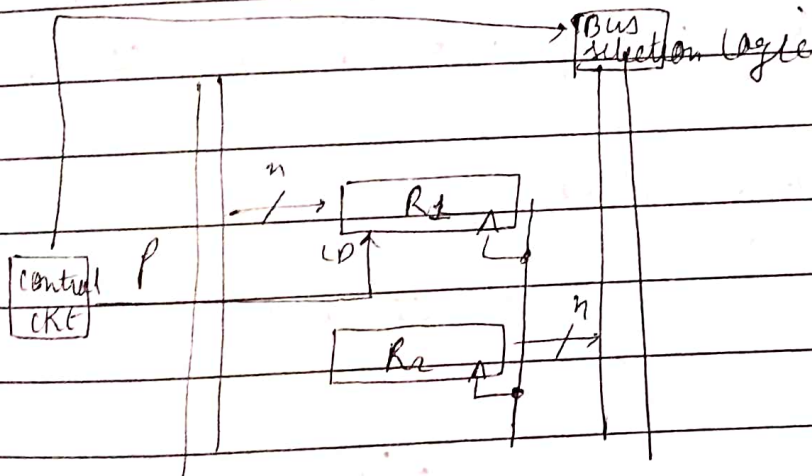
~~~~~



loading will happen at

rising edge of  $t+1$  be

only then uop is active.

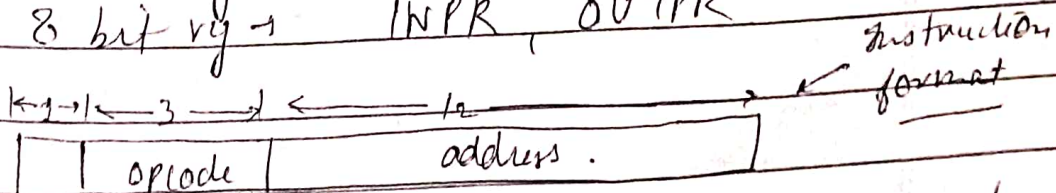
→ 1 memory unit  $4096 \times 16$ 

→ 16 bit registers → Accumulator (ACC)

IR, TR, DR

→ 12 bit reg → AR, PC

→ 8 bit reg → INPR, OUTPR



mode

address

one of the operand  
always lies in Acc.

AC  $\rightarrow$  Accumulator register

all operations are performed with memory operand & content of ~~operator~~ accumulator.

$\sim$  no. operand instruction

ex  $\rightarrow$  clear AC

complement AC

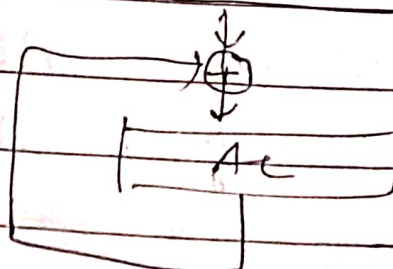
increment AC

for these types of instruction, the second part of instruction code can be used to specify ~~at~~ other operations of the computer.

# addressing Modes -

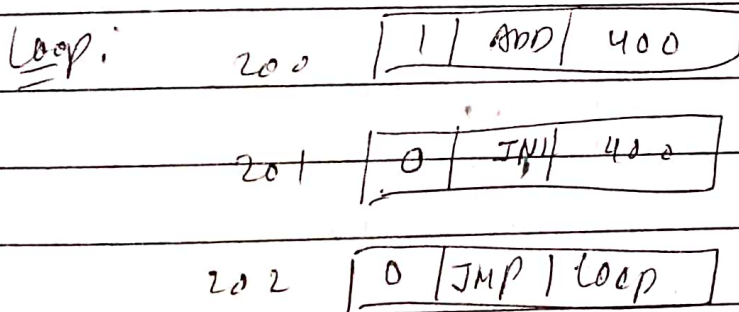
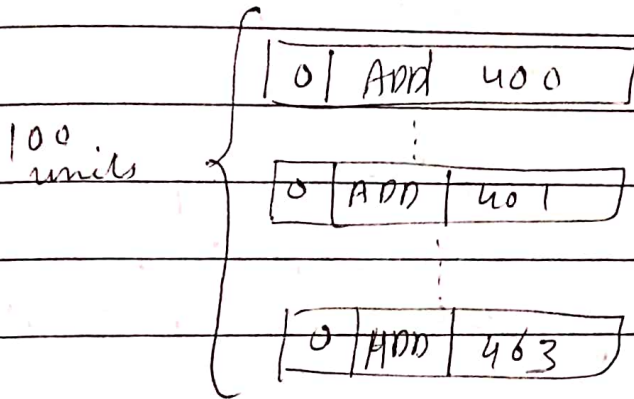
① direct addressing -

|     |   |         |     |
|-----|---|---------|-----|
|     | 0 | ADD     | 400 |
|     |   |         |     |
| 400 |   | operand |     |

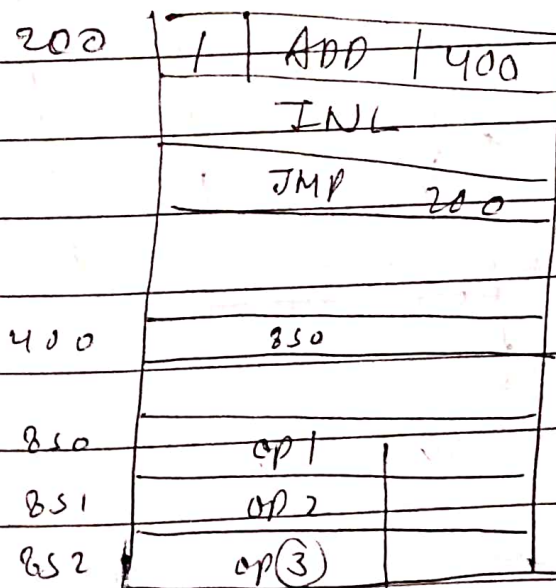


## Q. Indirect addressing -

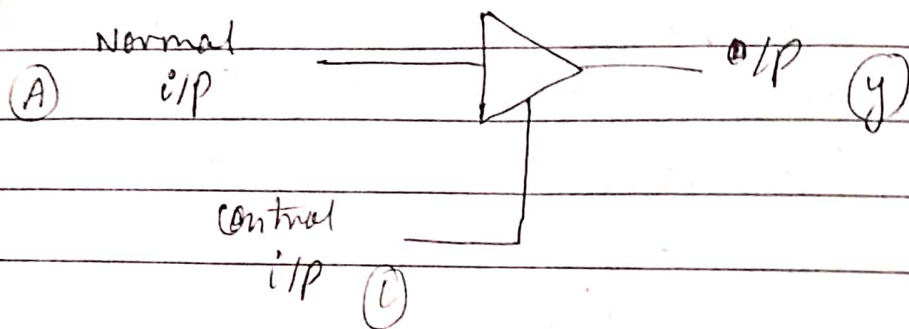
Suppose an array of 100 elements & we want to access array  
(eg. add it elements to AC)



~~00100000~~  
~~143~~

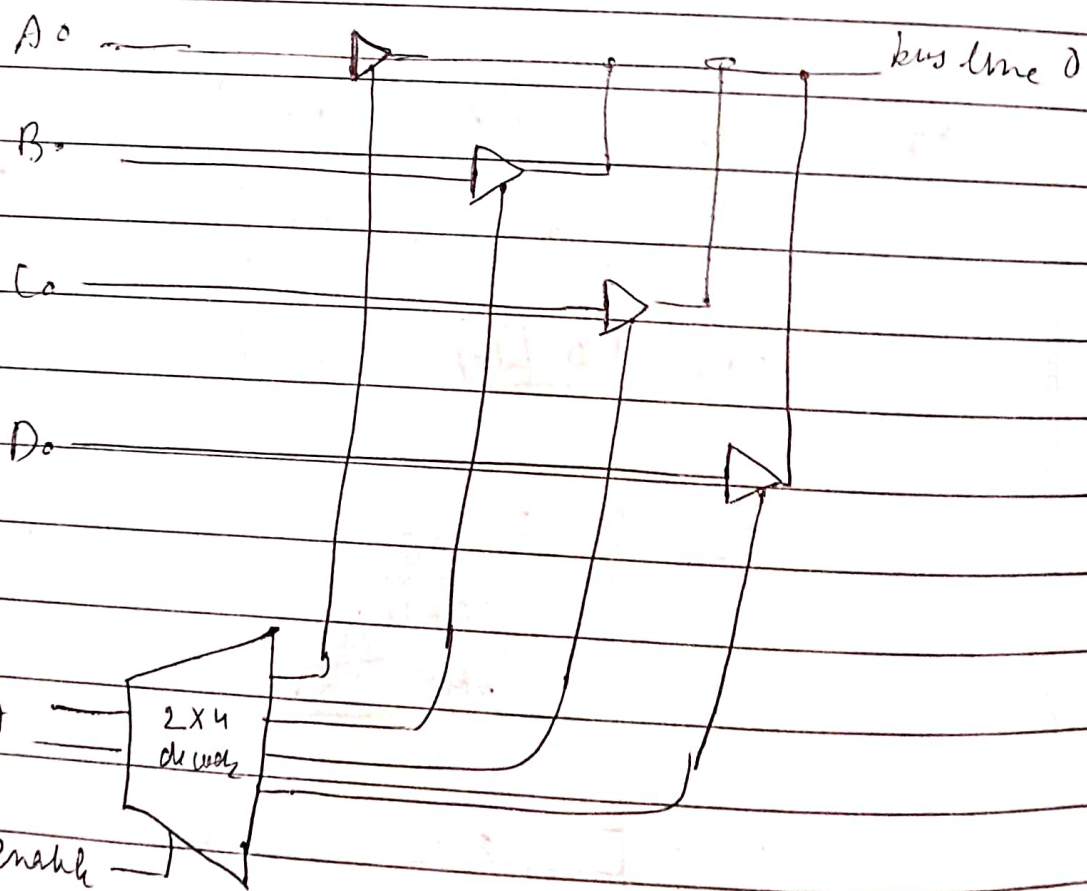


# # High Impedance Gate { 3-state buffer }



$$y = A \text{ (0 or } \pm \text{ if } c = 1)$$

~~$y = A$~~  or it will be high impedance state



They  
come  
from  
control  
ckt

Date: \_\_\_\_\_

Page No. \_\_\_\_\_

## Micro operations -

1. Register Transfer Mops -

$$P: R_1 \leftarrow R_2$$

2. Arithmetic mops

$$R_3 \leftarrow R_1 + R_2 \quad \text{ADD}$$

$$R_3 \leftarrow R_1 - R_2 \quad \text{SUB}$$

$$R_1 \leftarrow \overline{R_1} \quad \text{COMP}$$

$$R_2 \leftarrow \overline{R_2} + 1 \quad \text{2's comp.}$$

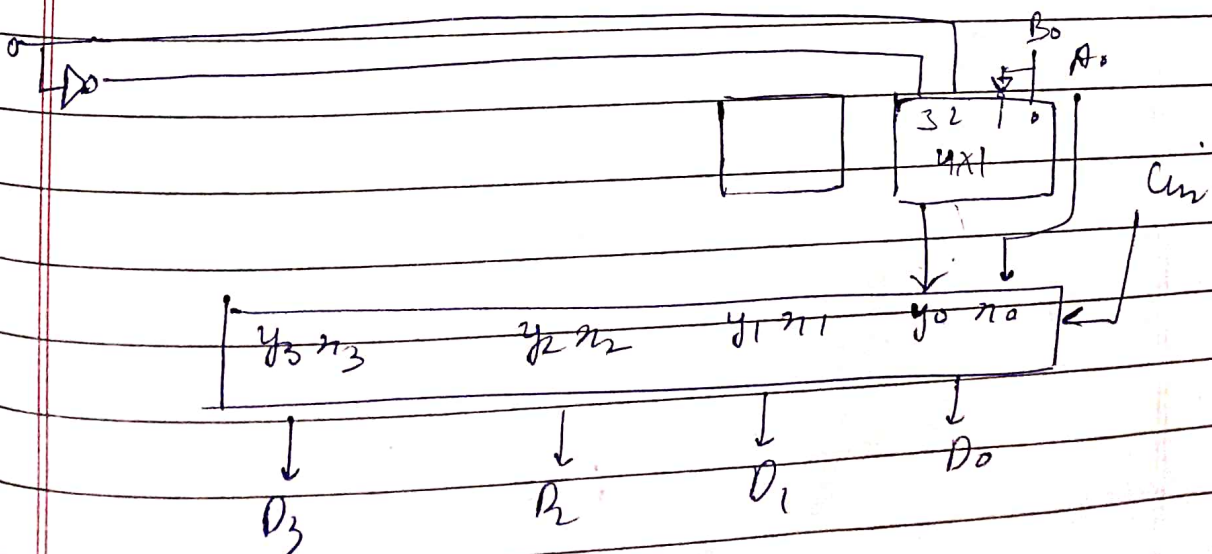
$$R_3 \leftarrow R_1 + \overline{R_2} + 1 \quad \text{SUB}$$

$$R_1 \leftarrow R_1 + 1 \quad \text{INC}$$

$$R_1 \leftarrow R_1 - 1 \quad \text{DCR}$$

3. Logic mops -

4. Shift mops



Date:

Page No.

i/p

o/p  $D = A + Y + C_{in}$ 

| $s_1$ | $s_0$ | $C_{in}$ | $Y$       | $O/P$                     | $D = A + Y + C_{in}$            |
|-------|-------|----------|-----------|---------------------------|---------------------------------|
| 0     | 0     | 0        | B         | $A + B$                   | (ADD)                           |
| 0     | 0     | 1        | B         | $A + B + 1$               | (ADD with carry)                |
| 0     | 1     | 0        | $\bar{B}$ | $A + \bar{B}$             | $(A - B - 1)$ (Sub with borrow) |
| 0     | 1     | 1        | $\bar{B}$ | $A + \bar{B} + 1 = A - B$ | (Sub)                           |
| 1     | 0     | 0        | 0         | A                         | Pass / transfer                 |
| 1     | 0     | 1        | 0         | $A + 1$                   | INC A                           |
| 1     | 1     | 0        | 1         | $A - 1$                   | Dec A                           |
| 1     | 1     | 1        | 1         | A                         | transfer                        |

~~0 0~~~~0 1~~~~1 0~~~~1 1~~

$$A + (1111) = A + (2^1 \text{ (or } 2) + 1) = \underline{\underline{A - 1}}$$

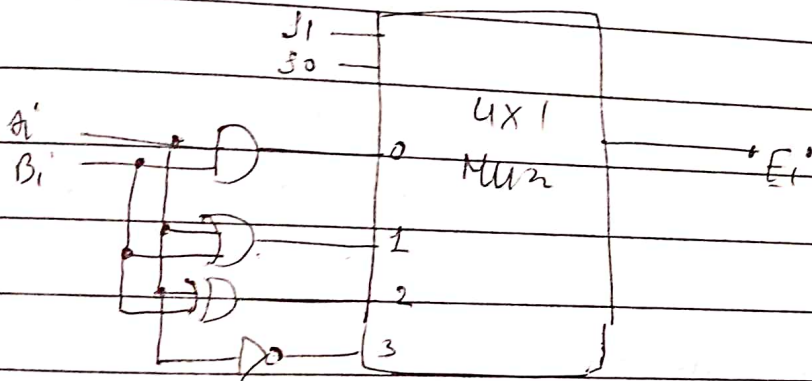
Date:

Page No.

# for 2 variables  $2^{2^2} = 16$  functions are possible.

| A | B | $F_0$ | $F_1$ | $F_{17}$ |
|---|---|-------|-------|----------|
| 0 | 0 | 0     | 0     |          |
| 0 | 1 | 0     | 0     | 1        |
| 1 | 0 | 0     | 0     | 1        |
| 1 | 1 | 0     | 1     | 1        |

we only consider AND, OR, XOR, complement



there will be  $n$ -such stages  
as  $n$ -bit logic circuit

| $S_1$ | $S_0$ | ops.             |
|-------|-------|------------------|
| 0     | 0     | $E = A \wedge B$ |
| 0     | 1     | $E = A \vee B$   |
| 1     | 0     | $E = A \oplus B$ |
| 1     | 1     | $E = \bar{A}$    |

Selective Set ~

(set the bit when mask is 1)

1 0 1 0 A

1 1 0 0 B

→ 1 1 1 0 A (i.e. A OR B)

Selective complement -

complement bits when mask is 1.

1 0 1 0 A

1 1 0 0 B

0 1 1 0 A (i.e. A XOR B)

Selective clear -

(pass when mask is 0)

1 0 1 0 (A)

1 1 0 0 (B)

0 0 1 0 (A) (i.e. A and  $\bar{B}$ )Mask - selective pass when mask is 1.

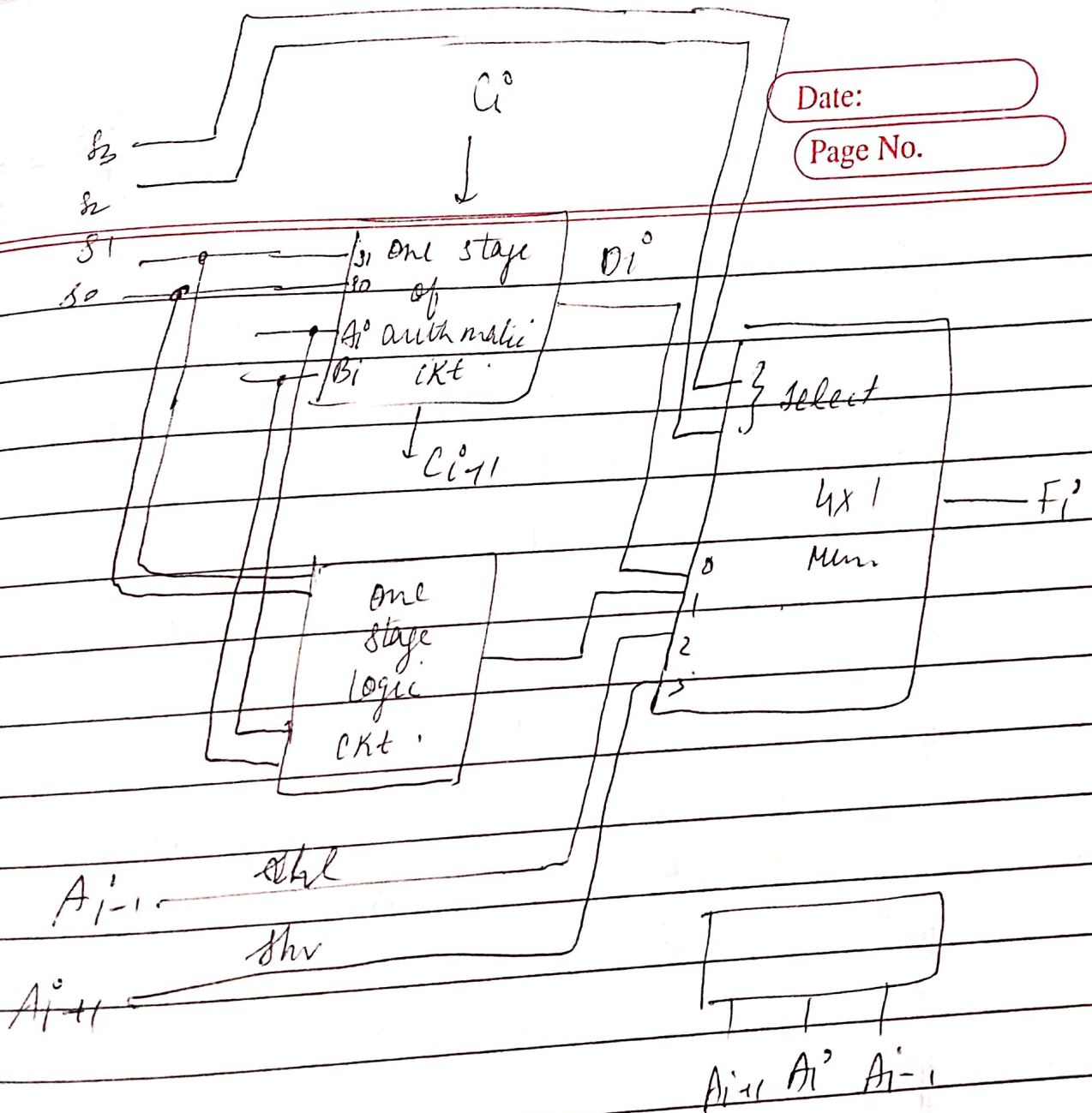
1 0 1 0 A

1 1 0 0 B

1 0 0 0 A (i.e. A AND B)

Date:

Page No.



| $S_3$ | $S_2$ | $S_1$ | $S_0$ | $C_{in}$ | O/P               |
|-------|-------|-------|-------|----------|-------------------|
| 0     | 0     | 0     | 0     | 0        | $A + B$           |
| 0     | 0     | 0     | 0     | 1        | $A + B + 1$       |
| 0     | 0     | 0     | 1     | 0        | $A + \bar{B}$     |
| 0     | 0     | 0     | 1     | 1        | $A + \bar{B} + 1$ |
| 0     | 0     | 1     | 0     | 0        | $A$               |
| 0     | 0     | 1     | 0     | 1        | $A + 1$           |
| 0     | 0     | 1     | 1     | 0        | $A - 1$           |
| 0     | 0     | 1     | 1     | 1        | $A$               |
| 0     | 1     | 0     | 0     | X        | $A \wedge B$      |
| 0     | 1     | 0     | 1     | X        | $A \vee B$        |
| 0     | 1     | 1     | 0     | X        | $A \oplus B$      |
| 0     | 1     | 1     | 1     | X        | $\bar{A}$         |
| 1     | 0     | X     | X     | X        | Shl A             |
| 1     | 1     | X     | X     | X        | Shr A             |

~ we can change inputs of shift to make it Arithmetic shift.

~ Through programming it can be done by masking & demasking operations.

# Instruction set -

HLL  $\rightarrow$  Compiler  $\rightarrow$  Machine level lang. ] Inst<sup>r</sup> set

is

Control  $\rightarrow$  ops.

① Arithmetic logic and shift

$\hookrightarrow$  ADD, CMA, INC  $\rightarrow$  SUB, MUL, DIV

② Logic

$\hookrightarrow$  AND, CMA, CLA

AND, CMA  $\rightarrow$  NAND

③ Shift

CIR, CIL

② Inst<sup>s</sup> for moving info. to and from memory and processor register.

STA, LDA

$\hookrightarrow$  STA 400  $\Rightarrow$  ~~A~~  $\rightarrow$  M[400]  $\leftarrow$  A

$\hookrightarrow$  LDA 400  $\Rightarrow$  A  $\leftarrow$  M[400]

3. Program control - branching & status check.

BUN 400  
(branches unconditionally)

BSA 350  
branch to 350 & save return address

LSZ  
Increment & skip if zero

SPA skip +ve accumulator

SNA (-ve)

SZA (zero)

SZE

SKI

skip input flag is set

SKE

skip output flag is set.

4. Input / output :

INP

OUT

2. Instruction set completeness -

A comp. should have a set of instructions so that the user can construct

Machine lay Program to evaluate any function that is known to be computable.

Our Instruction set is complete but it is not efficient bcz frequently used operations are not performed rapidly for ex  $\rightarrow$  sub, mul, div, etc.

## Instruction cycle -

- ① fetch inst<sup>n</sup> from Mem.
- ② Decode the inst<sup>n</sup>
- ③ Read effective address from memory if the inst<sup>n</sup> has an indirect address.
- ④ execute instruction

## # Fetch and decode -

$\sim$  PC is loaded initially with the address of first unit in the program

- IC cleared to zero, first timing signal in T<sub>0</sub> time signal go through the seq.

T<sub>0</sub>, T<sub>1</sub>, ...

2 ops for fetch & decode

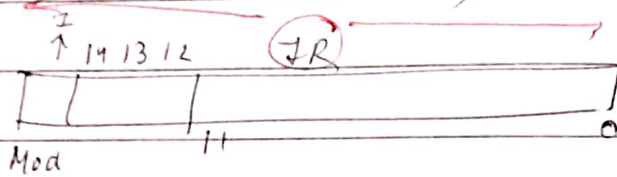
$T_0: AR \leftarrow PC$

$T_1: IR \leftarrow M[AR], PC \leftarrow PC + 1$

$T_2: D_0, D_1, \dots, D_7 \leftarrow \text{decode } IR(12-14)$

$AR \leftarrow IR(0-11)$

$I \leftarrow IR(15)$



## # Time & Control -

Master clock generator

~ reg is enable by a control signal.

~ control signals are generated in control unit.

~ control inputs for Mux in common bus.

- multiplexers

- process registers : INC, LD, CLR

- microops for acc.

~ Control organisation →

### (1) Hardwired Control -

Control logic implemented with gate, flip-flops, decoders & other digital CKTs.

~ It reqs. changes in wiring if designed is changed.

## ② Microprogrammed Control

A control ~~mem.~~ mem. is programmed to initial the seq. of ops.

~ Any req. changes or Modifications can be done by updating the microprogram in control memory.

AND

ADD

LDA

STA

BUN

BSA

ISZ

opcode

address

I=0

I=1

0000

xxx

xxx

xxx

0xxx

8xxx

4

4

4

1xxx

3xxx

2xxx

Axxx

3xxx

Bxxx

4xxx

Cxxx

5xxx

Dxxx

6xxx

Exxx

opcode

7 800

0111

1000 0000 0000

CLA

7 400

0111

0100 0000 0000

CLE

7 200

only 1 bit is 1

CMA

7 100

no need to use

CME

7 080

decoder

CIR

7 040

~ instead connect directly

CIL

7 020

bits

INC

7 010

SRA

7 008

SNA

SZA

7 004

82E

7002

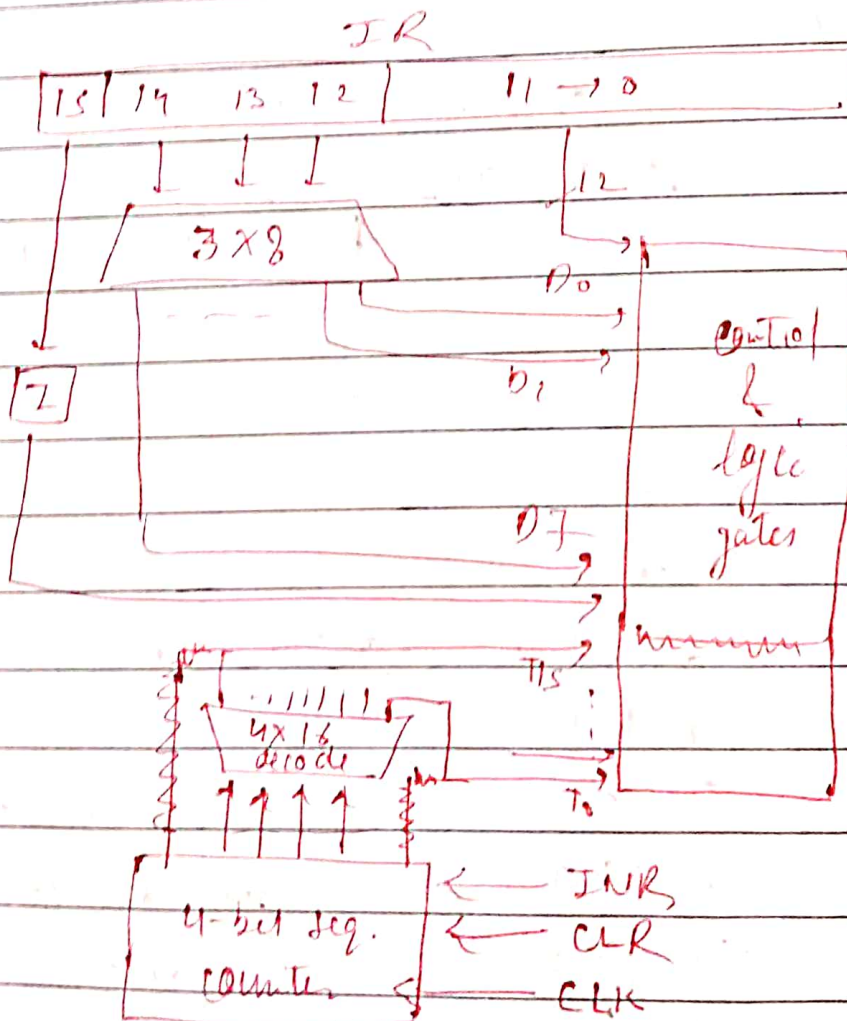
11LT

7001

7/0

707

|     |       |
|-----|-------|
| INP | F800  |
| OUT | F400  |
| SKT | F2004 |
| SKO | F100  |
| 70N | F080  |
| 70F | F040  |

0 → 2<sup>n</sup>

0 → 16

Date:

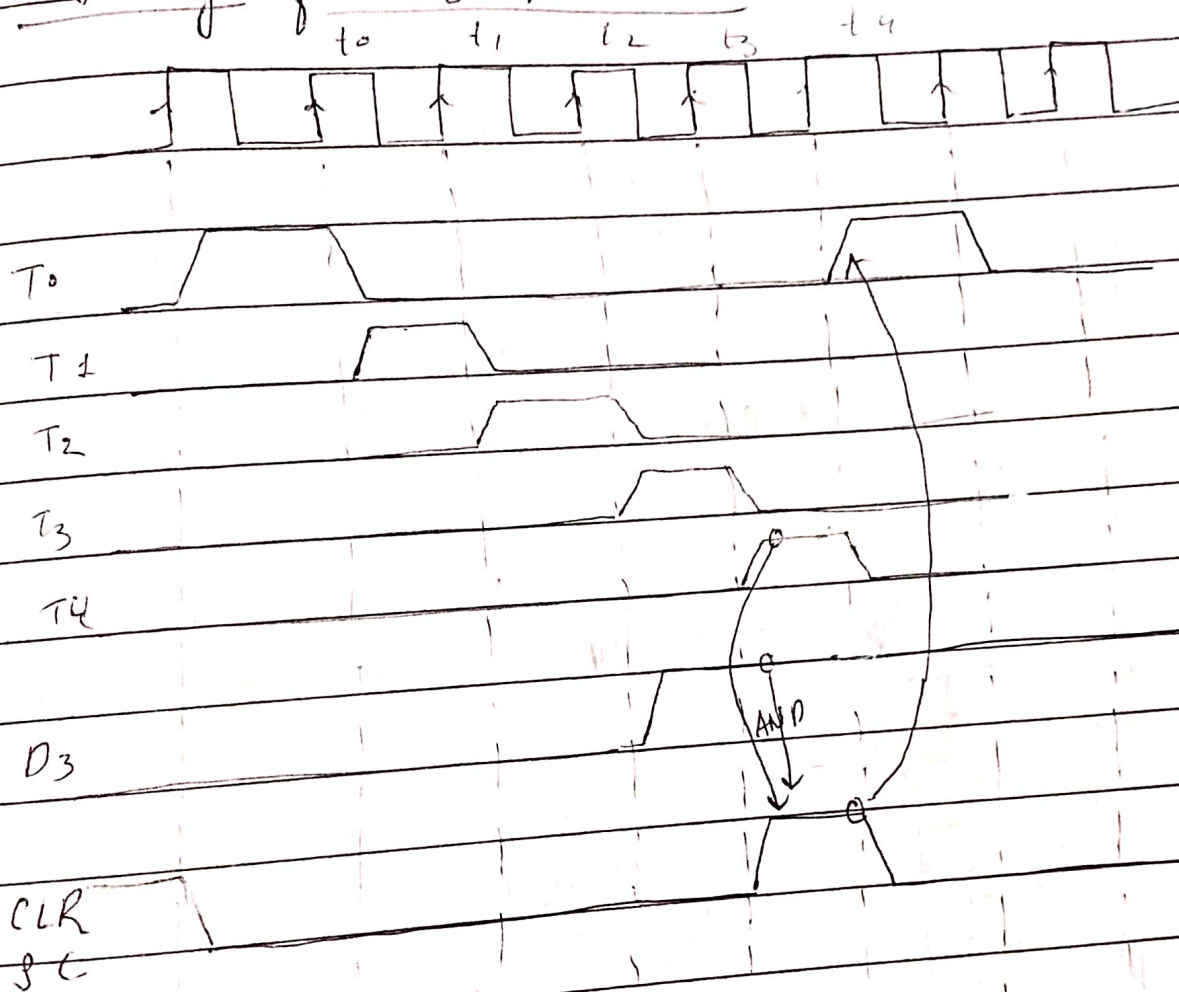
Page No.

R.R.L  $\rightarrow$  0111

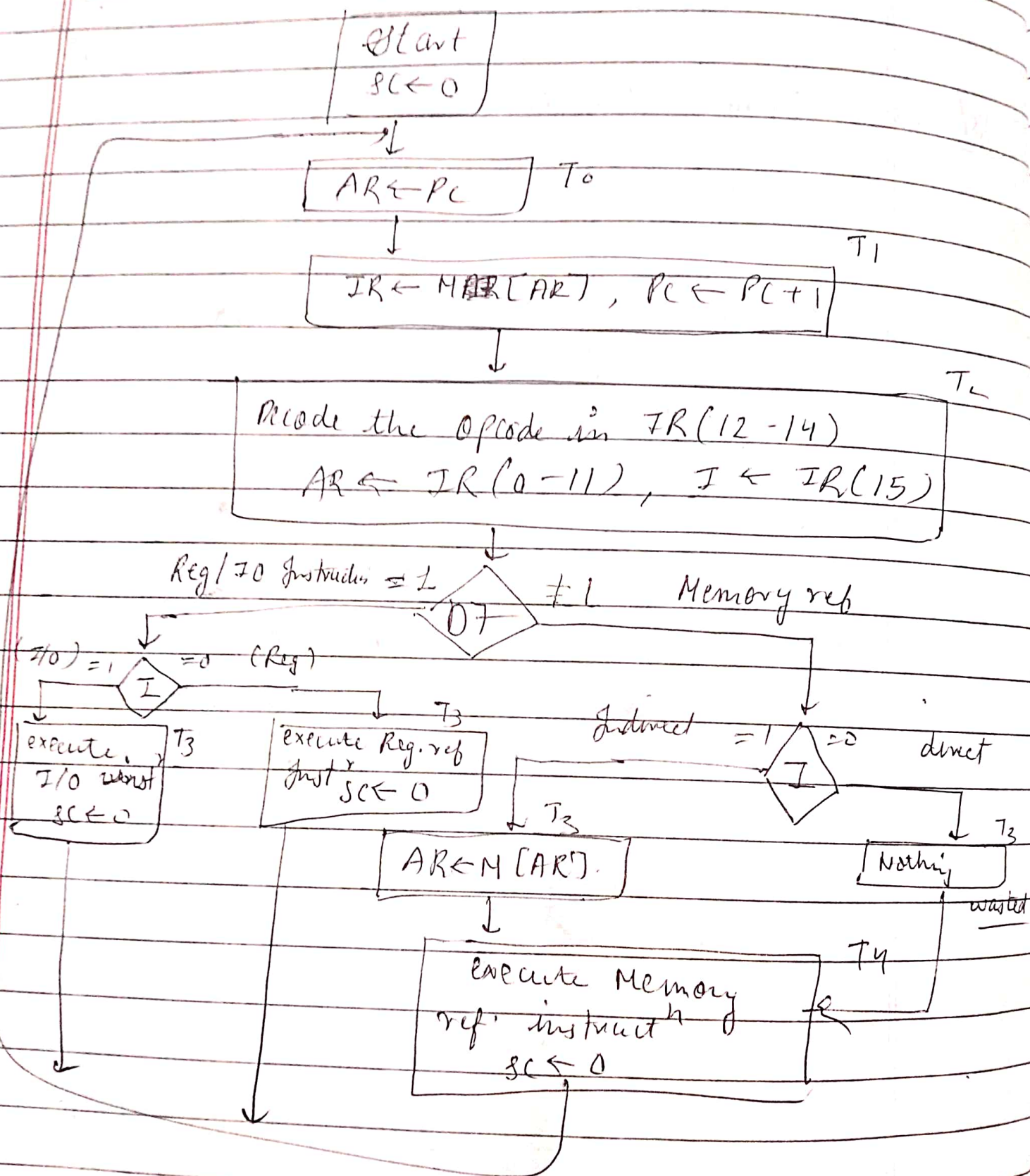
I/O  $\rightarrow$  1111

D7 will be high.

Timing for D3T4:  $SC \leftarrow 0$  -



Determine the type of Inst<sup>n</sup> -



The 4 paths are for 3 d/d type of Inst.  
The operation selected in each path is activated at the occurrence of clock transition where  $T_3$  is active.

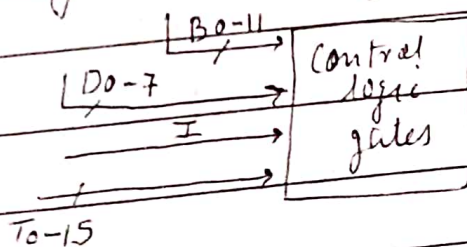
$D_7' \cdot IT_3 : AR \leftarrow M[AR]$

$D_7' \cdot I' \cdot T_3 : \text{Nothing}$

$D_7' \cdot I' \cdot T_3 : \text{execute a reg. reference Inst.}$

so, the reg. counter has to be incremented without doing anything when  $I=0$  for memory reference instructions (Direct addressing)

14/3/24. Control logic gates ~ other inputs



Inputs to control logic:

~ decoder outputs  $D_0 - D_7$

~ (S.C) decoder outputs  $T_0 - T_5$

OR(0-15) to check if  $DR=0$

value of  $F \cdot F_5 \rightarrow IEN, FGI, FGO$

$E, S, R, I$

- Bits 0 to 11 of  $IR$

-  $I$

- AC bits 0-15 to check if  $AL=0$  and sign bit  $AC(15)$

- ~ outputs of control logic circuit :
- signal to control i/p's (LD, INR, CLR) of nine registers.
  - To control the RD & WR i/p of memory.
  - to set, clear, or comp. the FFS.
  - signals  $S_0, S_1, S_2$  to select a register for the bus.
  - signals to control AC, address logic ckt.

#### # Control logic for fetch & decode

$T_0 : AR \leftarrow PC$

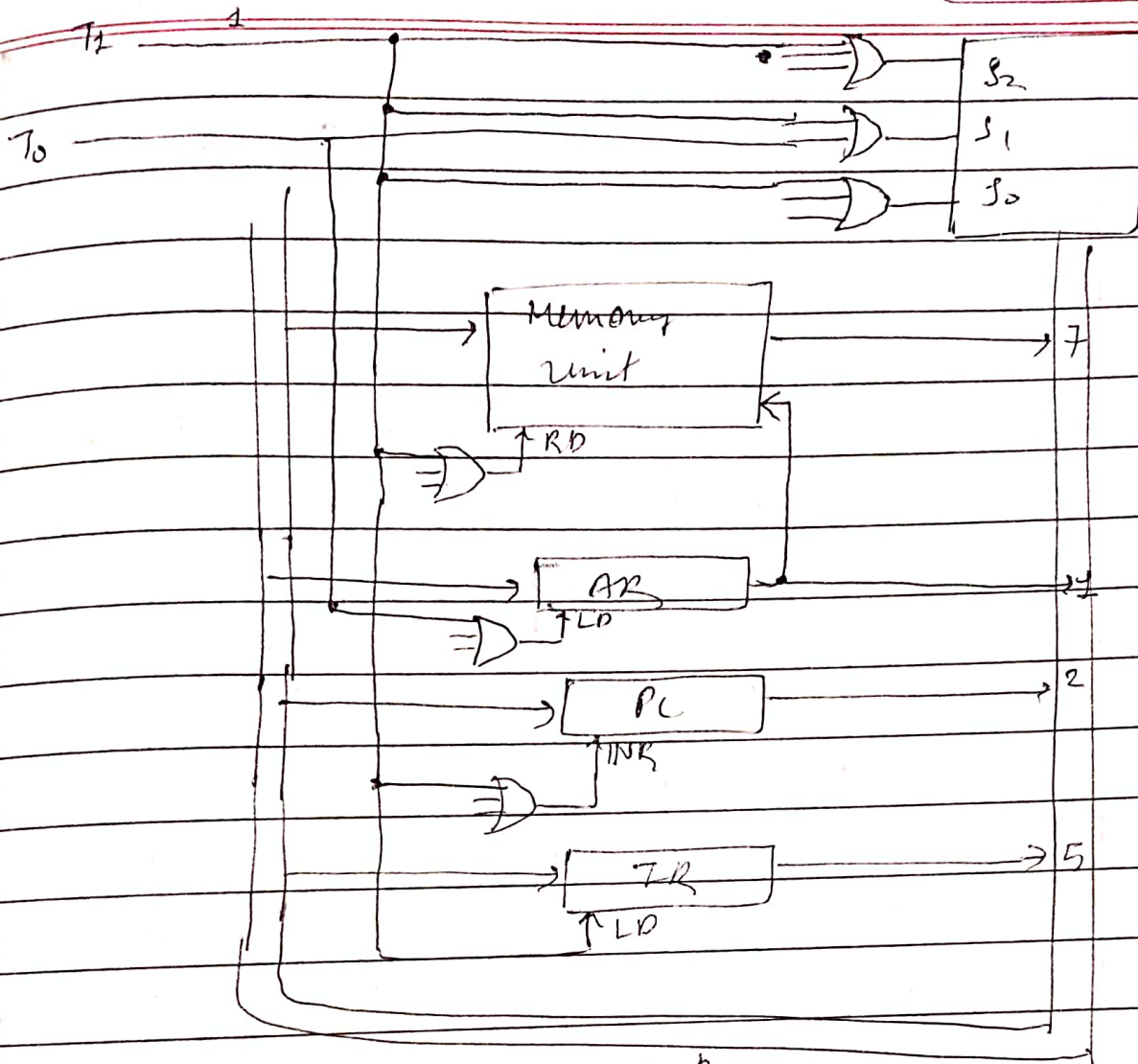
$T_1 : IR \leftarrow M[AR], PC \leftarrow PC + 1$

$T_2 : D_0 \dots D_7 \leftarrow IR(12-14)$

$AR \leftarrow IR(0-11), I \leftarrow IR(15)$

for  $T_1$  -

- ① enable the RD of m/m.
- ② Place the content of m/m onto the bus by selecting proper bus selection inputs ( $S_2, S_1, S_0 = 111$ )
- ③ Transfer the content of bus to IR by enabling LD i/p of IR.
- ④ Increment PC by enabling INR i/p of PC.



# Control logic for Reg. rty. Inst<sup>n</sup>

when  $D7 = 1$ ,  $I = 0$

- 12 reg. rty. Inst<sup>n</sup>. given by 12 bits of IR(0-11)
- $D7 \ I' T3 = 8$ .
- Bit  $i$  of IR is assigned symbol  $B_i$ .
- All control function can be denoted as

$$rB_i = D7 \ I' T3 B_i$$

~ eg: CLA = 7800

$$= \underbrace{0}_{I'} \underbrace{111}_{D7} \underbrace{1000}_{B11} \underbrace{0000} \underbrace{0000}$$

$$D7 \ I' T3 B_{11} = rB_{11} = 8 \text{ IR}(11)$$

→ refer more on 1<sup>st</sup> pg.

|      |     |                                                                            |
|------|-----|----------------------------------------------------------------------------|
|      |     | $\gamma: PC \leftarrow 0$                                                  |
| 7800 | CLA | $\gamma B_{11}: AC \leftarrow 0$                                           |
| 7400 | CLE | $\gamma B_{10}: E \leftarrow 0$                                            |
| 7200 | CMA | $\gamma B_9: AC \leftarrow \bar{AC}$                                       |
| 7100 | CME | $\gamma B_8: E \leftarrow \bar{E}$                                         |
| 7080 | CIR | $\gamma B_7: AC \leftarrow \text{shr}(AC), AC(15) \leftarrow E$            |
|      |     | $E \leftarrow AC(0)$                                                       |
| 7040 | CIL | $\gamma B_6:$                                                              |
| 7020 | INC | $\gamma B_5: AC \leftarrow AC + 1$                                         |
| 7010 | SPA | $\gamma B_4: \text{if } (AC(15) = 0) \text{ then } (PC \leftarrow PC - 1)$ |
| 7008 | SNA | $\gamma B_3: \text{if } (AC(15))$                                          |
| 7004 | SZA | $\gamma B_2:$                                                              |
| 7002 | SZE | $\gamma B_1:$                                                              |
| 7001 | HLT | $\gamma B_0:$                                                              |

15/3/23

Mem ref busexecution starts at  $T_4$ AND to AC $000 = D_0, AC \leftarrow AC \wedge M[AR]$ 

eg. AND 135

MOPS = $D_0 T_4: DR \leftarrow M[AR]$  $D_0 T_5: AC \leftarrow AC \wedge DR, PC \leftarrow 0$ ADD to AC $001 = D_1, AC \leftarrow AC + M[AR],$  $E \leftarrow \text{cout}$

D1T4:  $PR \leftarrow M[AR]$

D1T5:  $AC \leftarrow AC + PR$ ,  $PC \leftarrow PC + 1$ ,  $SC \leftarrow 0$

Date:

Page No.

LDA: Load to AC  $010 = D_2$

$AC \leftarrow M[AR]$

D2T4:  $OR \leftarrow M[AR]$

D2T5:  $AC \leftarrow PR$ ,  $SC \leftarrow 0$

This isn't done directly  
bcoz AC isn't directly  
connected to bus.

STA: Store AC  $011 = D_3$

$M[AR] \leftarrow AC$

D3T4:  $M[AR] \leftarrow AC$ ,  $SC \leftarrow 0$

BUN: (Branch unconditional)  $D_4 = 100$

eg. BUN 350 } Indicate we already has this  
address in AR

D4T4:  $PC \leftarrow AR$ ,  $SC \leftarrow 0$

• No fetch decode cycle

BSA (branch and save return address)

eg. BSA 135

D5T5:  $M[AR] \leftarrow PC$ ,  $AR \leftarrow AR + 1$

D5T5:  $PC \leftarrow AR$ ,  $SC \leftarrow 0$

PC  $\rightarrow$  49

PC  $\rightarrow$  50

PC  $\rightarrow$  51

PC  $\rightarrow$  550

BUN 350  
ADD 130

...

|   |             |     |
|---|-------------|-----|
| 0 | BSA         | 135 |
|   | next instr? |     |
|   |             |     |
|   |             |     |
|   |             |     |
| 1 | BUN         | 135 |

after execution of BSA

|   |            |     |
|---|------------|-----|
| 0 | BSA        | 135 |
|   | next instr |     |
|   |            |     |
|   |            |     |
|   |            |     |
| 1 | BUN        | 135 |

Memory, PC & AR at T4 before

Ex → D5T4:  $M[135] \leftarrow 21$ ,  $AR \leftarrow 136$

D5T5:  $PC \leftarrow 136$ ,  $SC \leftarrow 0$

Date:

Page No.

ISZ: (Increment & Skip if zero) - D<sub>6</sub>

eg. ISZ 450

D6T4:  $DR \leftarrow M[AR]$

D6T5:  $DR \leftarrow DR + 1$

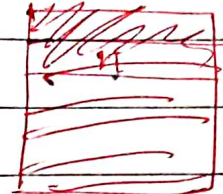
D6T6:  $M[AR] \leftarrow DR$ , if  $(DR = 0)$  then  $PC \leftarrow PC + 1$

#

1st 2nd  
2nd 2nd

COOP

also



ISZ 500

BON 120

19/3/24

#

Input-Output Instructions -

Computer registers  
2 F F5

I/O terminal

Serial communication

I/F

Printer

Receiver  
Interface

Sec 11-3

3e Mano

OUTR

AC

Keyboard

Transmitter  
Interface

INPR

0

F47

→ 1

• For  $FLI$ , initially it is cleared to zero. When a key is struck in keyboard, a char is shifted into  $INPR$  &  $FLI$  is set to 1. As long as  $FLI$  is 1, the info in  $INPR$  cannot be changed by striking another key. The comp. checks the flag bit. If it is 1 the char is transferred to  $AL$  and  $FLI$  is cleared to zero.

•  $FL0$ , initially it is equal to 1, comp. checks  $FL0$ , if it is 1 the info to be printed is transferred from  $AL$  to  $OUTR$  and  $FL0$  is cleared to zero. Output device after printing the info sets  $FL0$  equals to 1.

1111  
└──┘  
07

$$P = D_7 I T_3$$

F800 INP  $PB_{11}$  :  $AL(0-7) \leftarrow INPR, FLI \leftarrow 0$

OUT  $PB_{10}$  :  $OUTR \leftarrow AL(0-7), FL0 \leftarrow 0$

SKI  $PB_9$  : If  $(FLI = 1)$  then  $PC \leftarrow PC + 1$

SNO  $PB_8$  : If  $(FL0 = 1)$  then  $PC \leftarrow PC + 1$

ION  $PB_7$  :  $IEN \leftarrow 1$

IOF  $PB_6$  :  $IEN \leftarrow 0$

3E → 159 (5'-6)  
3E → 133 (5'-2)

Date:

Page No.

e.g. INPUT program: Input a char.

LOOP, SKI // check input flag.  
BUN LOOP // if Flag = 0 branch to check again.  
INP // flag = 1, input, char (transferred)  
INPR → AC

OUT // Print the char.

STA CHR // Store char.

HLT // Instruct

CHR, —

END // assembly language directive  
to stop job of assembler.

e.g. OUT char:

LDA CHR // load char to AC

loop, SKI // check flag.

BUN LOOP // if flag = 0, processor not ready, check again

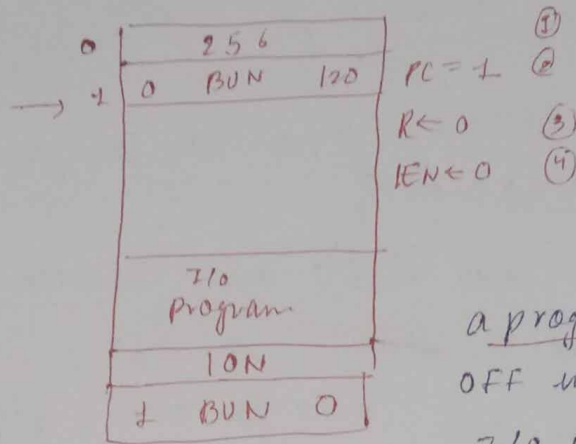
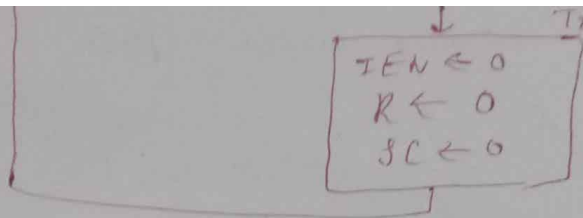
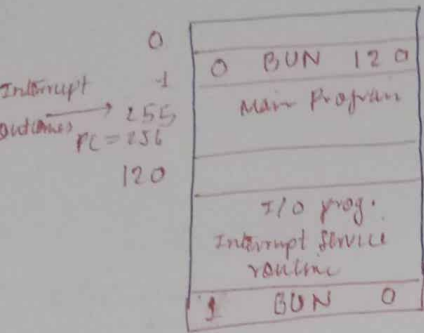
OUT // flag = 1

HLT

CHR, HEX 0057 // char is ASCII 'w'

END

## Memory Snapshot



a programmer should specifically  
off interrupt while going in  
I/O program & then enable  
at the end of this I/O prog.

Interrupt

[ T<sub>0</sub>' T<sub>2</sub>' T<sub>2</sub>' (IEN) (FGI + FG0): R ← 1

~ Interrupt cycle:

RT<sub>0</sub> : AR ← 0 , TR ← PC

RT<sub>1</sub> : M[AR] ← TR , PC ← 0

RT<sub>2</sub> : PC ← PC + 1 , IEN ← 0 , R ← 0  
SC ← 0

~ Fetch decode -

R'T<sub>0</sub> : AR ← PC

R'T<sub>1</sub> : IR ← M[AR] , PC ← PC + 1

R'T<sub>2</sub> : AR ← IR(0-11) . . . .

① Control of AR -

e.g. LD, INR, CLR .

∴ All statement that change content of AR (i.e AR ← . . .)

Fetch : R'T<sub>0</sub> : AR ← PC

Decode : R'T<sub>2</sub> : AR ← IR(0-11)

Indirect :  $\overline{O_7}$  IT<sub>3</sub> : AR ← M[AR]

Transfer to AO

RT<sub>0</sub> : Interrupt : AR ← 0 ] CLR.

~ The programs controlled transfer (polling / status check)

~ efficiency -

if inst<sup>n</sup> cycle time = 1  $\mu$ s

& print speed = 10 char/s

1 char in  $\frac{1}{10}$  s =  $10^5 \mu$ s.

~ want to find how many times the loop goes until to read 1 char?

check, SKO

BUN check

⇒ for status check, two inst<sup>n</sup>s are executed when comp. checks flag bits & decides not to transfer the info.

∴ every loop takes 2  $\mu$ s for checking flag each time

i.e.  $\frac{10^5 \mu\text{s}}{2 \mu\text{s}} = 50,000$  times the flag is checked.

# Interrupt →

Program

① The comp. doesn't continually check the flags.

② when a flag is set, the comp. is interrupted momentarily from ~~proceeding~~ proceeding into current program & is informed of the fact that the flag is set.

③ The comp. takes care of input/output transfer & returns to normal program after completion.

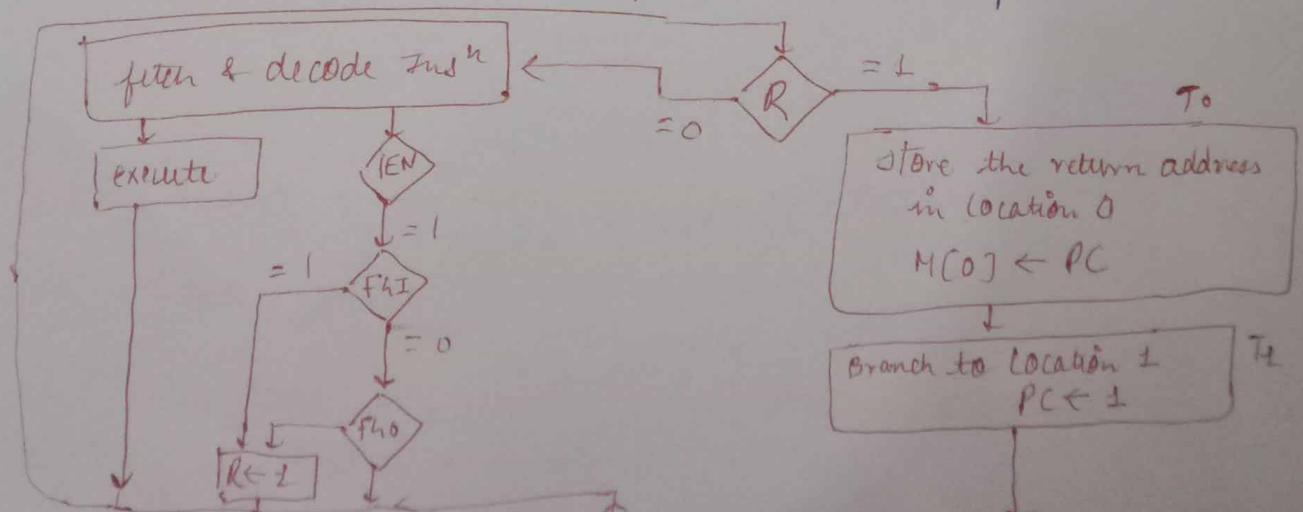
IEN (interrupt enable flip flop) - can be set or cleared with

2 instr<sup>n</sup>(s)      ION      ,      IEN ← 1

IOF      ,      IEN ← 0

when IEN = 0, flags interrupt the computer.

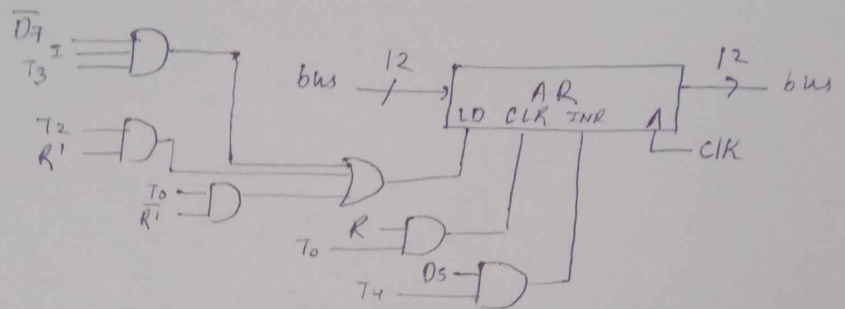
when IEN = 1, the comp. can be interrupted.



ISA:  $D5T4: AR \leftarrow AR + 1$  ] Increment.

$$LD(AR) = R'T_0 + R'T_2 + D_7'I T_3$$

$$CLK(AR) = RT_0, \quad \text{Incr}(AR) = D5T4$$



@ Memory (Read Input):

fetch:  $R'T_1: IR \leftarrow M[AR]$

Increment:  $\overline{D_7} I T_3: AR \leftarrow M[AR]$

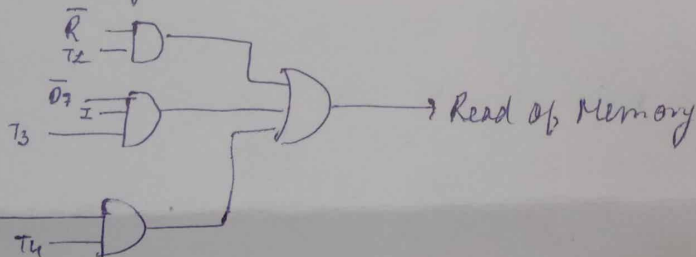
~~Interrupt~~ AND:  $D_0T_4: DR \leftarrow M[AR]$

ADD:  $D_1T_4: DR \leftarrow M[AR]$

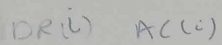
LDA:  $D_2T_4$

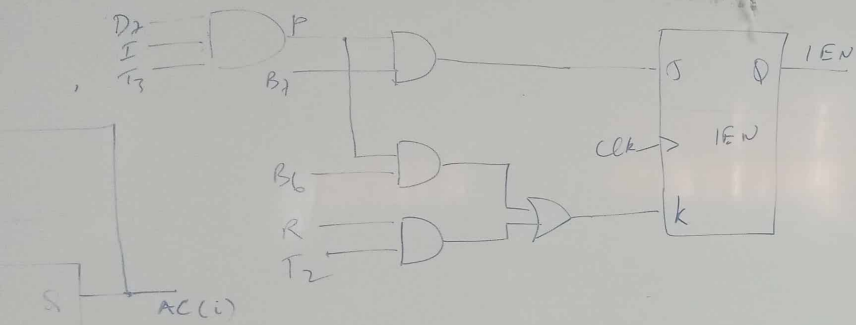
IS2:  $D_6T_4$

$$RD(Memory) = \overline{R}T_1 + \overline{D_7} I T_3 + (D_0 + D_1 + D_2 + D_6) T_4$$



Accumulator  $\leftrightarrow$  ALU - one stage





J K  $\text{En}$

|   |   |   |
|---|---|---|
| 0 | 1 | 0 |
| 1 | 0 | 1 |

Accumulator Logic

ADD  $AC \leftarrow AC + DR$   
(LHS)

